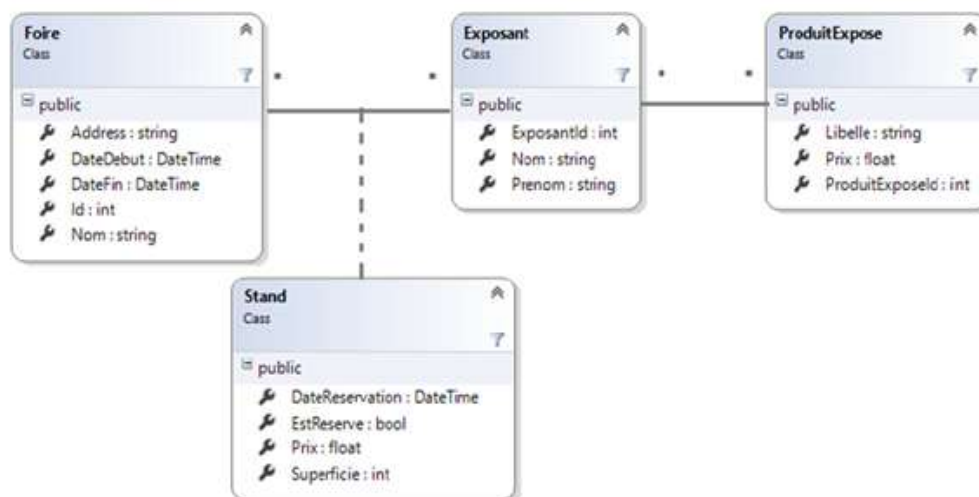


 esprit Se former autrement HONORIS UNITED UNIVERSITIES	EXAMEN Semestre : ☒ 1 ☐ 2 Période : ☒ 1 ☐ 2 Session : ☒ Principale ☐ Rattrapage
Module : Architecture des systèmes d'information I (.NET) Enseignants : Équipe .NET Classes : 4ALINFO7 Documents autorisés: ☐ OUI ☒ NON Calculatrice autorisée : ☐ OUI ☒ NON Nombre de pages : 8 pages Date : 27/10/2025 Heure : 09h00 Durée : 1h30m	
Nom et Prénom : Classe :	

Nous souhaitons créer une application qui permet la gestion d'un parc d'exposition d'une manifestation « Exposition Meuble ». Une représentation simplifiée de diagramme de classe est illustrée ci-dessous :



Partie I : Mise en place et conception orientée objet (6 pts)

- 1) Donner **trois arguments** pour convaincre votre client d'utiliser le Framework .Net pour l'implémentation de l'application souhaitée. **(0.75 pt)**

Back + front => framework complet
multi technologies (web, mobile, desktop, etc)
cross platform (multi platform: windows, linux, mac, ...)
Open source
.....
.....

- 2) Sachant que l'application sera accessible à une variété large d'utilisateurs, **quelle implémentation .Net doit être utilisée** pour le développement de l'application ?
Justifier votre réponse. (0.5 pt)

.Net Core / .Net 6 et + car ces implémentations sont multi-plateformes
.....

- 3) **Quelle architecture logicielle** proposez-vous pour l'implémentation de l'application ?
Justifier votre réponse. (0.5 pt)

origon / multi-couche / MVC / n-tiers => La séparation des préoccupatio
.....

- 4) En respectant l'architecture proposée en 3), énumérer **les espaces de noms** utilisés (**namespaces**) en indiquant le contenu de chacun. **(0.75 pt)**

Ex.Domain : Les entités
Ex.Application : Services
Ex.Data : couche d'accès à la BDD
Ex.Console

- 5) Implémenter le code de l'entité **Exposant** en incluant la ou les propriétés de navigation. (1.5 pt)

```
public class Exposant {  
.....  
public int ExposantId {get; set;}  
public string Nom {get; set;}  
public string Prenom {get; set;}  
.....  
public virtual IList<Stand> Stands {get; set;}  
public virtual IList<ProduitExpose> ProduitExposes {get; set;}  
}  
.....  
.....  
.....
```

- 6) Implémenter le code de l'entité **Stand** en incluant la ou les propriétés de navigation. (1.5 pt)

```
public class Stand {  
.....  
public DateTime DateReservation {get; set;}  
public bool EstReserve {get; set;}  
public float Prix {get; set;}  
public int Superficie {get; set;}  
.....  
public Foire MyFoire {get; set;}  
public Exposant MyExposant {get; set;}  
}  
.....  
.....
```

- 7) Utiliser le principe d'Encapsulation pour contrôler la propriété **DateFin** de la classe **Foire**. Ainsi cette propriété n'acceptera qu'une valeur cohérente avec celle de **DateDébut**. (0.5 pt)

```
DateTime df;  
public DateTime DateFin {get(){return df;}  
.....  
set(DateTime value){  
if(value >= DateDebut) df = value;  
return df  
}}  
.....
```

Partie II : Services métiers (4 pts)

- 8) Soit la classe de service **StandService**. Pour éviter le couplage entre cette classe et les autres classes, que proposez-vous ? (0.5 pt)

```
Associer une interface à la classe  
.....  
.....
```

9) Sachant que les données **Stand** sont chargées dans la propriété

IList<Stand> Stands {get ; set ;} de la classe **StandService**, Utiliser LINQ pour implémenter les services suivants :

- a) **GetStand** : Renvoie la liste des **Stand** réservés par un **Exposant** donné durant une période donnée. **(1 pt)**

```
IList<Stand> GetStand (Exposant ex, DateTime dd, DateTime df) .....  
{ ..return Stands.WHERE(s=>s.MyExposant.ExposantId == ex.ExposantId .....  
.....&& s.DateReservation >= dd .....  
.....&& s.DateReservation <= df) .....  
......TOLIST(); .....  
} .....
```

- b) **GetGainPrevu** : Renvoie le gain souhaité pour un **Exposant** donné durant une période donnée. **(1 pt)**

```
double GetGainPrevu(Exposant ex, DateTime dd, DateTime df) { .....  
.....return Exposant.ProduitExposes.SUM(p=>p.Prix) - .....  
.....GetStand(ex, dd, df).SUM(s=>s.Prix); .....  
} .....
```

- c) **GetSuperExposant** : Renvoie le (les) **Exposant** (s) ayant la plus grande superficie allouée durant une foire donnée. **(1.5 pt)**

```
IList<Exposant> GetSuperExposant (Foire f){ .....  
.....return Stands.WHERE(s=>s.MyFoire.Id == f.Id) .....  
.....GROUPBY(s=>s.MyExposant.ExposantId) .....  
.....ORDERBYDESCENDING(g=>g.SUM(s=>s.Superficie)) .....  
...... FIRST() .....  
......TOLIST(); .....  
} .....
```

Partie III : Entity Framework (10 pts)

- 10) Compléter la classe Context pour que chaque classe modèle soit mappée vers une table côté base de données. **(1.5 pt)**

```
public class FormationContext:DbContext {
    .....public DbSet<Foire> Foires {get; set;} .....
    .....public DbSet<Exposant> Exposants {get; set;} .....
    .....public DbSet<ProduitExpose> ProduitExposes {get; set;} .....
    .....public DbSet<Stand> Stands {get; set;} .....
    .....protected void OnConfiguring() .....
    .....{ .....
    .....}
}
```

- 11) Nous voulons configurer la clé étrangère "IdFoire" dans l'entité Stand compléter le code suivant : **(0.5 pt)**

```
public class Stand
{
    public int IdFoire { get; set; }
    .....[ForeignKey("IdFoire")] .....
    public virtual Foire Foire { get; set; }
```

- 12) Sachant que les propriétés DateDebut et DateFin de l'entité **Foire** et **DateReservation** de l'entité **Stand** sont de type Date, compléter le code suivant en incluant un message en cas de violation de cette contrainte. **(1 pt)**

```
public class Foire
{
    .....[DataType(DataType.Date, ErrorMessage="Entrez une date valide")] .....
    public DateTime DateDebut { get; set; }
    .....[DataType(DataType.Date, ErrorMessage="Entrez une date valide")] .....
    public DateTime DateFin { get; set; }
```

```
public class Stand
{
    .....[DataType(DataType.Date, ErrorMessage="Entrez une date valide")] .....
    public DateTime DateReservation { get; set; }
```

- 13) Sachant que la superficie d'un **Stand** doit être entre 6 et 20 m², compléter le code suivant pour ajouter cette contrainte. (0.5 pt)

```
public class Stand
{
    [Range(6,20)] .....
    public int Superficie { get; set; }
```

- 14) On veut que la longueur de toutes les colonnes de type « string » ne dépasse pas 150 caractères, compléter le code suivant: (1 pt)

```
protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)
{
    configurationBuilder.Properties<string>().HasMaxLength(150);
}
```

- 15) En utilisant une classe de configuration, on veut établir la relation entre **Foire**, **Exposant** et **Stand** afin que les clés étrangères de cette relation soient mappées vers des colonnes nommées **ExposantFKey** et **FoireFKey**, compléter le code suivant: (2 pts)

```
public class StandConfiguration : IEntityTypeConfiguration<Stand>
{
    0 références
    public void Configure(EntityTypeBuilder<Stand> builder)
    {
        // Configurer la cle étrangere ExposantFKey
        builder.HasOne(s => s.MyExposant)
            .WithMany(e => e.Stands)
            .HasForeignKey(s => s.ExposantFKey);

        // Configurer la cle étrangere FoireFKey
        builder.HasOne(s => s.MyFoire)
            .WithMany(f => f.Stands)
            .HasForeignKey(s => s.FoireFKey);
    }
}
```

- 16) La clé primaire de la classe **Stand** est composée des trois propriétés **ExposantFKey**, **FoireFKey** et **Datereservation**, compléter le code écrit dans la classe de configuration « StandConfiguration »: **(0.5 pt)**

```
public class StandConfiguration : IEntityTypeConfiguration<Stand>
{
    0 références
    public void Configure(EntityTypeBuilder<Stand> builder)
    {
        //clé primaire compose
        builder.HasKey (f => new
        { f.ExposantFKey, f.FoireFKey, f.DateReservation }
        );
    }
}
```

- 17) Pour bien appliquer cette classe de configuration, il faut l'appeler dans quelle méthode? **(0.5 pt)**

☐	<code>protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)</code>
☒	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code>
☐	<code>protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)</code>
☐	<code>protected override void HasConventions(ModelConfigurationBuilder configurationBuilder)</code>

- 18) On veut avoir une base de données nommée **FoireDb**, compléter le code de la chaîne de connexion: **(0.5 pt)**

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(@"Data Source=(localdb)\MSSQLLocalDB;
    Initial Catalog= FoireDb ;Integrated security =true");
}
```

- 19) Une migration a été créée, écrire la commande qui permet de créer la base de données. **(0.75 pt)**

`.Update-database`.....

20) Compléter le code suivant pour bien insérer un **Exposant** dans la base de données:

(0.5 pt)

```
FormationContext ctx = new FormationContext();
Exposant exposant = new Exposant
{
    Prenom="Ahmed",
    Nom="BEN AHMED"
};

ctx.Exposants.Add(exposant);
ctx.SaveChanges();
```

21) Pour bien configurer le « LazyLoading » qu'est-ce qu'il faut ajouter dans ce code:

(0.75 pt)

```
protected override void OnConfiguring ... (DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder
        .UseLazyLoading ... Proxies () .UseSqlServer (
```

Bon travail 😊